Name : Nilesh N.Swami

PRN: 260840320088

Q1.

```
package QueueSatck;

import java.util.Scanner;

public class QueueUsingTwoStacks {

		private int stackA[];
		private int stackB[];
		private int capacity;
		private int size;
		private int top;
		public QueueUsingTwoStacks(int capacity) {
			this.capacity = capacity;
			this.stackA = new int[capacity];
			this.stackB = new int[capacity];
			this.size = 0;
			this.top=-1;
		}

		public void enqueue(int x) {
			if(size == capacity-1) {
				System.out.println("stack is full");
				return;
			}
			stackA[++top] = x ;
//			stackB[top]=stackA[top];
			size++;
		}

		public int dequeue() {
			if(size == 0) {
				System.out.println("EMPTY");
				return -1;
			}
			int value = stackA[top--];

//			stackA[top--];
			size--;
			return value;
		}
```

```
	public int  peek() {
		if(size == 0) {
			System.out.println("EMPTY");
			return -1;
		}
		stackB[top]=stackA[top];
		int value = stackB[top];
		return value;
	}

	public int  size() {
		return this.size;
	}

public static void main(String[] args) {
	Scanner scan = new Scanner(System.in);
	System.out.println("Enter the capacity.");
	QueueUsingTwoStacks sq = new QueueUsingTwoStacks(scan.nextInt());

	char choice;

	do {
		System.out.println("Please choose your option.");
		System.out.println("E.push");
		System.out.println("D.pop");
		System.out.println("P.peek");
		System.out.println("S.size");
		System.out.println("B.exit");

		choice = scan.next().charAt(0);

		switch (choice) {
		case 'E':
//			int n = scan.nextInt();
//			for(int i=0;i<n;i++) {
				sq.enqueue(scan.nextInt());
//			}
			break;
		case 'D':
				int value = sq .dequeue();
				System.out.println("Dequeu:"+value);
			break;
		case 'P':
			int peekValue = sq.peek();
			System.out.println("Peaak Value :"+ peekValue);
			break;
```

```
                        case 'S':
                                int result  = sq.size();
                                System.out.println("Size : "+result);
                                break;
                        case 'B':
                                break;
                        }

                } while (choice != 'B');

        }

}
```

Output:

```
Enter the capacity.
7
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
E
How many elements do you want to push.
10
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
E
How many elements do you want to push.
20
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
E
How many elements do you want to push.
30
Please choose your option.
E.push
```

```
D.pop
P.peek
S.size
B.exit
D
Dequeu:30
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
E
How many elements do you want to push.
40
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
D
Dequeu:40
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
P
Peaak Value :20
Please choose your option.
E.push
D.pop
P.peek
S.size
B.exit
```

Q2.

```
package BinarySearchTree;

import java.util.Scanner;

public class BSTInorder {
```

```
static class Node{
        int key;
    Node left;
    Node right;


    Node(int key) {
       this.key = key;
       this.left = null;
       this.right = null;
    }
}

Node root;


BSTInorder(){
        this.root=null;
}



public static Node insert(Node root,int key) {

        if(root == null) {
                Node node = new Node(key);
                return node;
        }

        if(key<root.key) {
                root.left = insert(root.left,key);
        }else if(key>root.key) {
                root.right = insert(root.right,key);
        }
        return root;
}

public static void inorder(Node root) {
        if(root == null) {
                System.out.println("Tres is empty");
                return;
        }

        inorder(root.left);
        System.out.println(root.key+" ");
        inorder(root.right);
}
```

```
	public static void main(String[] args) {

		Scanner scan = new Scanner(System.in);
		BSTInorder bstinorder = new BSTInorder();
		System.out.println("How many node do you wan to create.");
		int num = scan.nextInt();
		for(int i=0;i<num;i++) {
			insert(bstinorder.root,scan.nextInt());
		}


		inorder(bstinorder.root);



	}

}
Output:
```

Q3.

```
package LinkedList;

import java.util.Scanner;

public class LinkedListDuplicates {

	static class Node{
		int data;
		Node next;

		Node(int data){
			this.data=data;
			this.next = null;
		}
	}

	Node head;
	Node tail;

	public void insert(int data) {
		Node newNode = new Node(data);
		if(head==null) {
			head = newNode;
```

```
                        tail = newNode;
                        return ;
                }
                tail.next = newNode;
                tail = newNode;
        }

        public void printDuplicates() {
                if(head == null) {
                        System.out.println("List is empty");
                        return ;
                }

                Node tempNode = head;
                int count = 0;
                while(tempNode.next != null) {

                        int  temp = tempNode.data;

                        tempNode =tempNode.next;
                        if(temp == tempNode.data) {
                                count++;
                        }
                }
                System.out.println("Count "+ count);
        }
        public static void main(String[] args) {

                Scanner scan = new Scanner(System.in);
                System.out.println("How mnay elment add.");
                int num = scan.nextInt();
                LinkedListDuplicates list = new LinkedListDuplicates();
                for(int i=0;i<num;i++) {
                        list.insert(scan.nextInt());
                }

                list.printDuplicates();
        }

}
```